Lctr 17: Bitwise Instructions

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Bitwise operations in C.
- Bitwise logic instructions having C counterparts.
- Bitwise logic instructions having no C counterparts.

Demo project:

- Various ASM functions using bitwise operations.

17.1 Introduction

After this lecture, you should be able to see how the C bitwise operations are implemented using the ASM bitwise logic instructions efficiently.

For comparison convenience, we reproduce the table of C's bitwise operators below.

Precedence	Operator	Description	Associativity
2	~	Bitwise NOT	Right to left
8	&	Bitwise AND	Left to right
9	^	Bitwise XOR	Left to right
10	|	Bitwise OR	Left to right
14	&=, ^=, |=	Assignment by bitwise operation	Right to left

17.2 Demo project

We use a demo project, `fi1s_logic_instructions`, to illustrate a number of ASM functions and their unit tests.

17.2.1 Abbreviations used in exemplary code and demo code

- Motivational applications (MTA) of bitwise operations.
- Illustration of the principle of bitwise logic (PBL) operations based on `uint8_t` variables.
- A short requirements (REQ) for writing the functions.
- A C implementation function (CFN) that satisfies the requirements.
- An ASM implementation function (AFN) that satisfies the requirements.
- Unit test functions (UTF) used to test the C and ASM functions under test (FUT).
- Code annotation (COA). We focus on code annotation for the AFN.

17.3 ASM bitwise instructions

Instruction	Description
`MVN Rd, Op2`	Logic NOT: Rd = ~Op2
`AND {Rd,} Rn, Op2`	Logic AND: Rd = Rn & Op2
`ORR {Rd,} Rn, Op2`	Logic OR: Rd = Rn | Op2
`EOR {Rd,} Rn, Op2`	Logic EOR: Rd = Rn ˆ Op2
`ORN {Rd,} Rn, Op2`	Logic NOT OR: Rd = Rn | ~Op2
`BIC {Rd,} Rn, Op2`	Bit clear (logic NOT AND): Rd = Rn & ~Op2
`BFC Rd, #s, #w`	Bit field clear: $b_{Rd,w+s-1} \ldots b_{Rd,s} = 0$
`BFI Rd, Rn, #s, #w`	Bit field insert: $b_{Rd,w+s-1} \ldots b_{Rd,s} = b_{Rn,w-1} \ldots b_{Rn,0}$

17.4 Application examples of bit-based operations

17.4.1 Checking a bit from a variable or register

A	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
mask = 1 << N	0	0	1	0	0	0	0	0
A & mask	0	0	b_5	0	0	0	0	0

Figure 1: Reading a bit using AND with a mask.

17.4.1.1 Reading a bit from a variable

MTA: We simulate reading a GPIO pin to check if the key connected to this pin is pushed or not without being affected by and affecting the other peripherals on the other pins on the same GPIO port.

REQ: Write a function in C and ASM, respectively, to **read bit `N` of a variable and return a Boolean value**.

CFN:

```c
bool mp_read_a_bit_c(uint32_t A, int N) {
    A = (A >> N) & 1U;
    bool bit_N = (bool)A;
    return bit_N;
}
```

AFN:

```
@ bool mp_read_a_bit_s(uint32_t A, int N)
mp_read_a_bit_s:
    lsr     r0, r1    @ A >>= N
    and     r0, #1    @ A &= 1U
    bx      lr
```

17.4.1.2 Reading the C flag

MTA: We want to read the C flag from the APSR without being bothered by the other bits of APSR.

PBL: By using the AND operation with a mask, we can easily read the C flag, bit 29 from APSR, as shown in Figure 2.

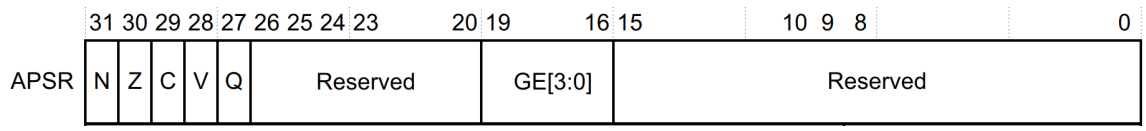

Figure 2: Reading a bit using AND with a mask.

REQ: Write a function in ASM that can **return the value of the C flag as a bool value**.

AFN:

```
@ bool mp_read_c_flag(void)
mp_read_c_flag:
    mrs     r0, apsr    @ move the value of apsr to r0
    lsr     r0, #29
    and     r0, #1
    bx      lr
```

COA:

- We read the value of APSR to R0 using `mrs r0, apsr`, where MRS stands for *Move to (general-purpose) Register from Special-purpose register*.
- The C flag is bit 29 of the APSR. Hence we need to shift the value of R0 to the right by 29 bits so that the C flag is at the LSB of R0.
- Next, we do bitwise AND operation of R0 with 1 using the `and r0, #1` instruction to *mask out* all the other bits.
- The return from the function will be 1 if the C flag is set; otherwise, it will be 0.

17.4.2 Setting a bit of a variable or register to 1

A	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
mask = 1 << N	0	0	1	0	0	0	0	0
A |= mask	b_7	b_6	1	b_4	b_3	b_2	b_1	b_0

Figure 3: Setting a bit using OR with a mask.

REQ: Write a function in C and ASM, respectively, to **set bit `N` of a variable to 1 without changing the other bits**.

CFN:

```c
uint32_t mp_set_a_bit_c(uint32_t A, int N) {
    uint32_t mask = 1U << N;
    A |= mask;
    return A;
}
```

AFN:

```
@ uint32_t mp_set_a_bit_s(uint32_t A, int N)
mp_set_a_bit_s:
    ldr     r2, =1      @ mask = 1U
    lsl     r2, r1      @ mask <<= N
    orr     r0, r2      @ A |= mask
    bx      lr
```

UTF:

```c
void test_set_bit1_using__set_a_bit_s(void) {
    uint32_t A = 0xDD;   // A   = 0b1101_1101
    uint32_t exp = 0xDF; // exp = 0b1101_1111, bit 1 set
    uint32_t act = mp_set_a_bit_s(A, 1);
    TEST_ASSERT_EQUAL_INT32(exp, act);
}
```

17.4.3 Clearing a bit of a variable or register to 0

A	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
~mask	1	1	0	1	1	1	1	1
A &= ~mask	b_7	b_6	0	b_4	b_3	b_2	b_1	b_0

Figure 4: Clearing a bit using AND with an inverted mask.

REQ: Write a function in C and ASM, respectively, to **clear bit `N` of a variable to 0 without changing the other bits**.

CFN:

```c
uint32_t mp_clear_a_bit_c(uint32_t A, int N) {
    uint32_t mask = 1U << N;
    A &= ~mask;
    return A;
}
```

AFN:

```
@ uint32_t mp_clear_a_bit_c(uint32_t A, int N)
mp_clear_a_bit_s:
    ldr     r2, =1      @ mask = 1U
    lsl     r2, r1      @ mask <<= N
    bic     r0, r2      @ A &= ~mask
    bx      lr
```

UTF:

```c
void test_clear_bit2_using__clear_a_bit_s(void) {
    uint32_t A = 0xDD;   // A   = 0b1101_1101
    uint32_t exp = 0xD9; // exp = 0b1101_1001, bit 2 cleared
    uint32_t act = mp_clear_a_bit_s(A, 2);
    TEST_ASSERT_EQUAL_INT32(exp, act);
}
```

17.4.4 Toggling several bits of a variable or register

A	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
mask = 0x28	0	0	1	0	1	0	0	0
A ^= mask	b_7	b_6	$\bar{b}_5$	b_4	$\bar{b}_3$	b_2	b_1	b_0

Figure 5: Toggling several bits using EOR with a multi-bit mask. $\bar{b}_n$ is the inverse of b_n.

REQ: Write a function in C and ASM, respectively, to **toggle a number of bits indicated by a mask without changing the other bits**.

CFN:

```
uint32_t mp_toggle_bits_c(uint32_t A, uint32_t mask) {
    A ^= mask;
    return A;
}
```

AFN:

```
@ uint32_t mp_toggle_bits_s(uint32_t A, uint32_t mask)
mp_toggle_bits_s:
    eor     r0, r1      @ A ^= mask
    bx      lr
```

UTF:

```
void test_toggle_bits4n5_using__toggle_bits_s(void) {
    uint32_t mask = 0x30;   // mask = 0b0011_0000
    uint32_t A = 0xDD;      // A    = 0b1101_1101
    uint32_t exp = 0xED;    // exp  = 0b1110_1101
    uint32_t act = mp_toggle_bits_s(A, mask);
    TEST_ASSERT_EQUAL_INT32(exp, act);
}
```

17.5 Application examples of field-based operations

17.5.1 Assigning several bits of a variable or register

MTA: Many times we need to assign a number of consecutive bits of a variable or register without affecting the other bits. This is an extension of setting or clearing a single bit discussed before. For example, to set up the mode of an GPIO pin, we need to assign two bits in the mode control register.

PBL: To assign a number of consecutive bits **systematically**, we need to do it in two steps:

- Step 1. Clear these bits, which will be assigned, by using a mask.
- Step 2. Assign the bits to the cleared field using an OR operation.

A	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
~mask	1	1	0	0	0	1	1	1
A &= ~mask	b_7	b_6	0	0	0	b_2	b_1	b_0
v = mask & (v << s)	0	0	1	0	1	0	0	0
A |= v	b_7	b_6	1	0	1	b_2	b_1	b_0

Figure 6: Assigning several bits.

REQ: Write a function in C and ASM, respectively, to **assign a number of consecutive bits indicated by a mask without changing the other bits**. There are a few parameters to be used:

- A variable `A` with which a field will be assigned on.
- A variable `s` used to indicate the start of the bits (0 for bit 0).
- A variable `w` used to indicate the width of the bits (2 for 2 bits).
- A variable `v` used to indicate the value to be assigned to A.

CFN:

```c
uint32_t mp_assign_bits_c(uint32_t A, int s, int w, uint32_t v) {
    uint32_t mask = (1 << w) - 1;
    mask <<= s;
    A &= ~mask;
    A |= mask & (v << s);
    return A;
}
```

AFN:

```
@ uint32_t mp_assign_bits_c(uint32_t A, int s, int w, uint32_t v)
@ r0 = A, r1 = s, r2 = w, r3 = v
mp_assign_bits_s:
    push    {r4, lr}    @ save R4 for the caller
    ldr     r4, =1      @ mask = 1U
    lsl     r4, r2      @ mask = 2^w
    sub     r4, #1      @ mask = w 1s
    lsl     r4, r1      @ the final mask
    bic     r0, r4      @ clear bits
    lsl     r3, r1      @ prepare the value to assign
    and     r3, r4      @ ensure the value to assign has w bits
    orr     r0, r3      @ assign the value
    pop     {r4, pc}    @ equivalent to bx lr
```

UTF:

```
void test_assign_bits3to5_using__assign_bits_s(void) {
    int s = 3;              // Start bit
    int w = 3;              // Width of bit field
    uint32_t v = 5;         // V   = 0b101 for assignment
    uint32_t A = 0xDD;      // A   = 0b11011101
    uint32_t exp = 0xED;    // exp = 0b11101101
    uint32_t act = mp_assign_bits_s(A, s, w, v);
    TEST_ASSERT_EQUAL_INT32(exp, act);
}
```

COA for UTF:

- The value of `A` is chosen as `0b1101_1101`, as shown before.
- We want to assign bits 3 and 5 to 5 = 0b101. So the value of `exp` is then `0b1110_1101`= `0xED`.

COA for AFN:

- The values of the first 4 registers are: R0 = `0b1101_1101`, R1 = 3, R2 = 3, and R3 = 5 = 0b101. They are passed by the caller.
- We need to use R4 for operation, so we push it to the stack; we push LR to the stack as well.
- We form the mask to be used in the function.
  - First, `mask` should be `0b0111` for three bits. This is done in lines from `ldr` to `sub`.
  - Then, we shift the mask to the left by 3 bits using the `lsl` instruction.
- We clear the 3 bits using the `bic` instruction. Here is the calculation we perform:

```
    0b1101_1101
BIC 0b0011_1000
---------------
    0b1100_0101
```

- We then shift the value in R3 to the left by `s` bits and AND it with the mask so the value in R3 should be `0b0010_1000`. Note that we use the AND operation here to just make sure a mistake in the value of `v` will not affect any other bits; so this is just a precaution.
- We finally use the OR operation to *assign* the intended bits.

17.6 Additional exercises

Exercise 1. Consider the code below (assume it works):

```
uint8_t A = 0b1101_1101;    // can be arbitrary
uint8_t M = 0b0000_0111;    // this is the mask
uint8_t B = 0b0000_0100;    // 3 bits to assign
uint8_t C = ~M;
uint8_t D = A & C;
A = D | B;
```

1. Determine the value of `C`, `D`, and `A` after running the above code.
2. If our ultimate goal is to calculate `A`, how can we write a single line of C code to replace the last three lines given above.
3. Translate each of the last three lines of code into ASM instructions assuming `A`, `M`, `B`, and `C` are saved in R0, R1, R2, and R3, respectively. You can use other registers as needed.
4. What will be the value of `A` if we replace the last three lines code by using `A |= B;`.
5. What will be the values of `E` and `F` if we add two more line of code as below? Any pattern you can see from the value of `F`?

```
uint8_t E = B ^ M;
uint8_t F = E + B;
```